

Problem statement, page 1 *what v2 covers (green = done, amber = partly)*

Advanced AI Engineering Project

Automated ESG Greenwashing Detection & Verification Agent

1. Background & Business Context

Environmental, Social, and Governance (ESG) compliance has become a critical pillar for corporate valuation and risk assessment. However, a major risk vector currently emerging is "Greenwashing"—the practice where organizations **make exaggerated or misleading claims** about the environmental benefits of their products, operations, or investments.

Manually auditing corporate sustainability reports, PR statements, and supply chain disclosures against real-world data (news, regulatory filings, environmental watchdog reports) is virtually impossible at scale. Businesses and regulatory bodies urgently need intelligent systems that can automatically cross-reference corporate claims with external realities to highlight discrepancies and reputational risks.

2. Project Objective

Design and build an **Agentic AI Verification System** that ingests a company's sustainability reports, extracts key ESG claims, and autonomously searches external public sources to verify or contradict these claims. The system should generate a **"Greenwashing Risk Score"** alongside an explainable audit trail detailing the evidence.

3. Technical Architecture & Requirements

The project requires a **modular, agent-based approach** rather than a single monolithic model. The pipeline should include:

3.1 1. Claim Extraction Engine

- Utilize a **Large Language Model** (e.g., Llama-3 or Mistral) to parse corporate sustainability PDFs or text.
- Implement **prompt engineering** to accurately extract falsifiable claims (e.g., "We reduced carbon emissions by 40% in 2023" or "Our supply chain is 100% deforestation-free").

3.2 2. Autonomous Verification Agent (Retrieval-Augmented Generation)

- Build an intelligent agent using frameworks like **LangChain** or CrewAI.
- The agent must take the extracted claims and construct search queries to retrieve external context from mock databases, news APIs, or public environmental datasets.
- **Contradiction Detection:** The LLM must analyze the retrieved external text against the corporate claim to determine if they **Align**, **Contradict**, or lack sufficient evidence.

3.3 3. Scoring & Explainability Pipeline

- Develop an algorithmic mechanism to aggregate the contradictions into a single **Greenwashing Risk Score (0-100)**.

1 **DONE** Exaggerated / misleading claims

We plant 15 greenwashing types: inflated %, baseline shift, scope swap, unretired RECs, hidden penalty, fake zero-deforestation...

2 **DONE** Reports + PR vs real-world data, at scale

Reads 25-30 pg PDFs. Checks vs BRSR filings, NGT/SPCB orders, CPCB OCEMS, satellite forest alerts, REC registry, news. Company PR = tier 4, filings outweigh it.

3 **DONE** Ingest, extract, search, score, audit trail

Upload PDF, 46/46 planted claims found, agents query 8 sources live, Risk Score 0-100, every tool call + decision logged with time.

4 **DONE** Modular, agent-based

LangGraph: 11 agents, 1 file each. Each claim runs its own sub-graph in parallel (Send API).

5 **PARTLY** LLM (Llama-3/Mistral) parses PDF + prompt eng.

PyMuPDF parses, Jev (typed decision model) picks claims, prompts versioned. Grog/Llama slot wired but OFF tonight (mock), so not Llama doing it.

6 **DONE** LangChain + build queries + mock DBs/news/env data

LangGraph (LangChain family). Decomposer splits claim, router picks sources, investigator hits mock_sources API (REST + MCP). Never hardcoded.

7 **DONE** Align / Contradict / Insufficient

Jev reads claim + evidence + calculator output, gives verdict with probabilities. Decision model, not a chat LLM: no maths in the model.

8 **DONE** Algorithmic 0-100 risk score

Logistic model on Jev outputs, weights fitted on 140 train cases (AUC 0.99). Vajra 56 High, Sahyadri 45, Kaveri 34, unknown co 24.

Problem statement, page 2 *deliverables + how we get judged*

9

- Ensure the system outputs a transparent report containing the original claim, the external evidence found, and the reasoning behind the specific risk flag. ✓

4. Expected Deliverables

10

1. A clean, well-documented Python repository containing the agentic orchestration code. ✓
2. A small, curated dataset (can be synthetically generated) of corporate claims and corresponding external news/reports used to test the system. ✓

11

3. A lightweight user interface (using Streamlit or Gradio) where a user can upload a text document and view the real-time extraction, agent reasoning, and final risk score. ~

12

4. A concise (2-3 page) technical summary explaining the agent design, prompt methodologies used, and limitations of the system. ✓

5. Evaluation Criteria

13

- **Agentic Logic:** How well the agent breaks down complex claims into searchable queries. ✓

14

- **Accuracy & Hallucination Control:** The system's ability to accurately detect contradictions without making up external evidence. ✓

15

- **Code Modularity:** The use of clean, object-oriented, or functional programming paradigms to separate the extraction, retrieval, and evaluation steps. ✓

16

- **Practical Utility:** The clarity and business-readiness of the final explainable report output. ✓

9

DONE Transparent report

Per claim: text + page, evidence cards (tier + API endpoint), calc steps, reasoning. Report tab + full audit trail.

10

DONE Repo + curated synthetic dataset

appl/graph = orchestration. 150 cos, ~9.75K rows / 10 tables, 500 news articles, 200 labelled claims, 4 report PDFs.

11

PARTLY UI (Streamlit or Gradio)

Built in React instead: live agent trace (SSE), drill-down, verify-a-claim. Old Streamlit still runs, if jury insists.

12

DONE 2-3 pg technical summary

docs/TECHNICAL_SUMMARY (md + docx): agent design, prompt method, results, honest limitations.

13

DONE Agentic logic

Claim breaks into sub-claims, each typed + routed. Shown live in the trace.

14

DONE Accuracy + hallucination control

200 cases: 98%. No retrieval 35%, news-only RAG 45%. Relevance gate, citation check, abstains when no proof.

15

DONE Code modularity

agents / tools / graph / ingest / decision split. 247 tests pass offline.

16

DONE Business-ready output

Company score, top red flags with page refs, PDF report. Honest co not flagged.

Scorecard: where we stand

14 of 16 marked points fully done, 2 partly. Nothing missing.

Done (14)

- Claim extraction from real-looking 23-28 pg PDFs: 46/46
- Agents fetch evidence live over API (8 sources), REST + MCP
- Align / Contradict / Insufficient with probabilities
- Fitted 0-100 risk score + top red flags
- Audit trail of every step, live on screen
- LangGraph multi-agent, modular code
- Synthetic dataset: 150 cos, ~9.75K rows, 200 test claims
- 2-3 pg technical summary
- Benchmark 98%, ablations prove retrieval matters
- 247 tests, offline replay

Partly (2)

- **#5 LLM = Llama/Mistral:** decisions by Jev, Groq/Llama slot exists but was off. Fix: put Groq key, set LLM_PROVIDER=groq, rerun once.
- **#11 Streamlit/Gradio:** we chose React. Streamlit v1 still runs as backup.

Say it before they ask

- All data synthetic; test score 100% is optimistic (we saw test errors while fixing).
- Chart-image claims not read, only text + tables.
- Triage tool, not a legal verdict.

✓ green = done ~ amber = partly